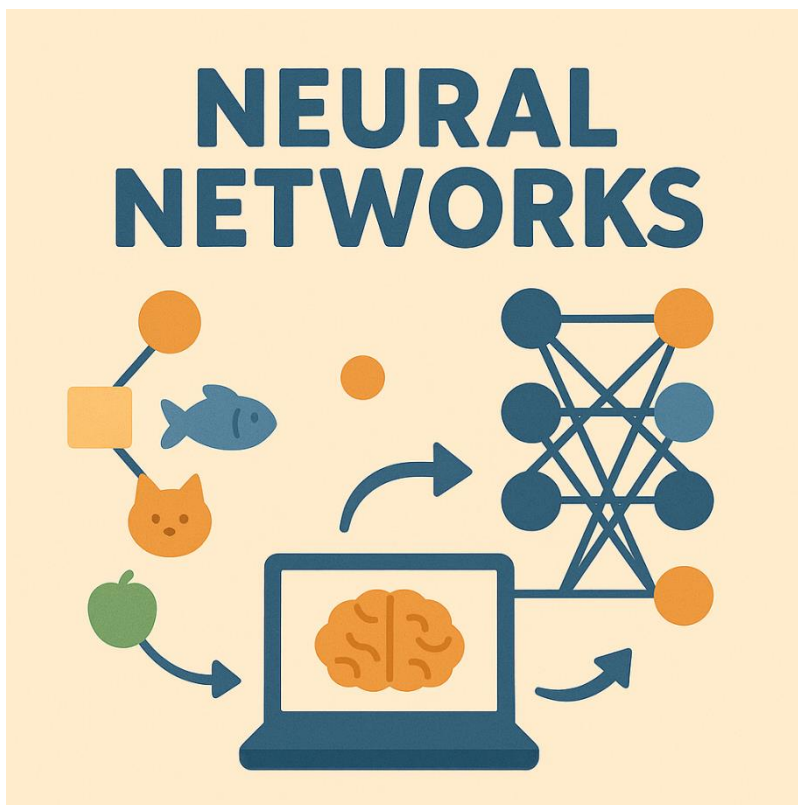


NF4 – XARXES NEURONALS



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Introducció.....	4
Neurona biològica.....	5
Perceptró	6
Bias (biaix)	7
Procés d'entrenament d'un perceptró	7
Xarxa neuronal.....	9
Arquitectura bàsica.....	10
Funció d'activació	11
Funció pas (step)	12
Funció sigmoide / logística.....	12
Funció Tanh (tangent hiperbòlica).....	13
Funció ReLU (R ectified L inear U nit)	14
Funció Leaky ReLU	14
Funció Softmax.....	15
Quadre resum de les funcions d'activació	17
.....	18
Resum	18
Entrenament	20
Forward propagation	20
Funció de cost (loss function)	21
Backpropagation (retropropagació de l'error)	22
Gradient descent (Descens de gradient)	22
Optimització i actualització de pesos.....	23
Regularització.....	24

Preprocessat	25
Disseny de l'arquitectura	25
Hiperparàmetres	25
Frameworks de Xarxes Neuronals	27
TensorFlow	27
PyTorch	27
Keras	28
Exeprimentació amb TensorFlow	28
Implementació d'una xarxa neuronal amb Keras	29
Exemple de Xarxa Neuronal - Regressió	29
Tipus de Xarxes Neuronals	31
FNN /MLP (Feedforward Neural Network /Multilayer Perceptron)	31
CNN (.....	32
RRNN (.....	32
LSTM	33
GANs	33
Xarxes Autoencoder	33
Transfer Learning	Error! No s'ha definit el marcador.
Annex	Error! No s'ha definit el marcador.
Referències	35

Introducció

La Natura ha estat, al llarg de la història, una font d'inspiració constant per a la humanitat. Moltes de les gran innovacions tecnològiques no han sorgit del no-res, sinó de l'observació atenta dels mecanismes i estratègies que la natura ha perfeccionat durant milions d'anys d'evolució. Aquest camp s'anomena **biomimètica**, i consisteix a estudiar sistemes naturals per replicar-ne els principis en solucions tecnològiques.

Per exemple l'aviació moderna es va inspirar en els ocells; els sistemes de sonar i radar tenen el seu origen en l'ecolització dels ratpenats; el famós velcro utilitzat en els vestits espacials i inspirat a la flor del card especialitzada en enganxar-se a la roba o al pèl dels animals.

Aquesta mateixa lògica és la que va donar lloc al naixement de les xarxes neuronals artificials, un dels pilars fonamentals del Machine Learning modern (**Deep Learning**). Les xarxes neuronals són models computacionals inspirats en el funcionament del cervell humà format per xarxes de neurones biològiques. Alguns investigadors no els hi agrada parlar d'aquesta analogia biològica per la gran diferència que hi ha en el funcionament d'una neurona biològica i un **perceptró** (neurona artificial).

És cert que el funcionament dels dos elements és totalment diferent, però a mi m'agrada l'analogia amb el fet de que el treball d'una sola neurona no té cap "valor", però el treball conjunt de moltes neurones permet la realització de funcions extraordinàriament complexes i és aquí a on l'analogia té sentit. **Moltes unitats simples que , treballant conjuntament, són capaces d'aprendre patrons complexos a partir de dades.**

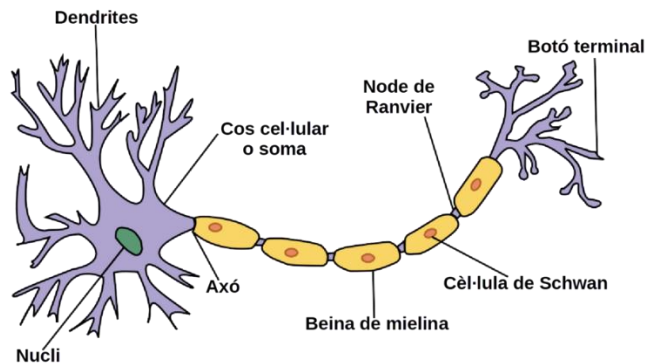
Les **xarxes neuronals** s'han convertit en una eina molt eficaç d'aprenentatge automàtic perquè són **capaces d'adaptar-se a molts problemes (regressió, classificació, llenguatge natural, imatges, so,...)**

El concepte de Xarxa Neuronal apareix per primera vegada en el 1943 de la mà del neuropsicòleg Warren McCulloch i el matemàtic Walter Pitts, en el seu treball de referència "[A Logical Calculus of Ideas Immanent in Nervous Activity](#)".

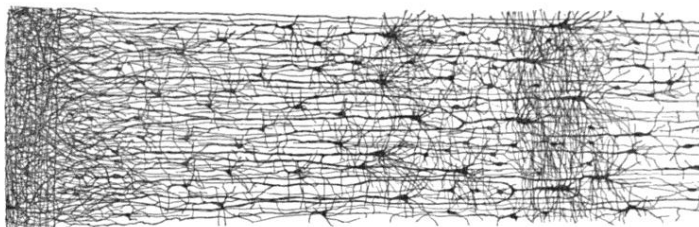
A la imatge tenim en Warren a l'esquerra i en Walter a la dreta.



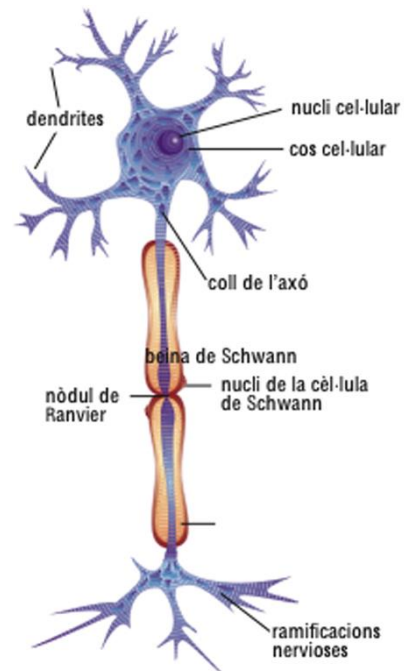
Neurona biològica



Wikipèdia



Art de la citoarquitectura del còrtex humà. Wikipèdia



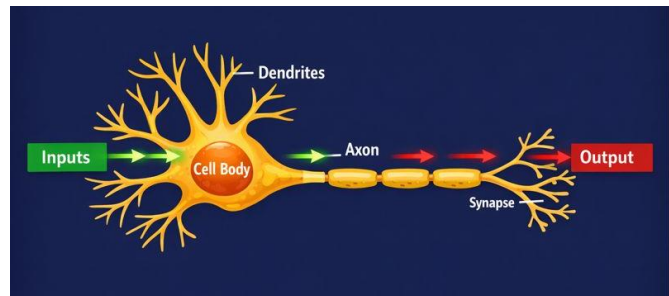
Enciclopèdia Catalana (Enciclopedia.cat)

Les parts bàsiques d'una neurona biològica són: **dendrites**, **axó** i les **ramificacions nervioses** o botons terminals.

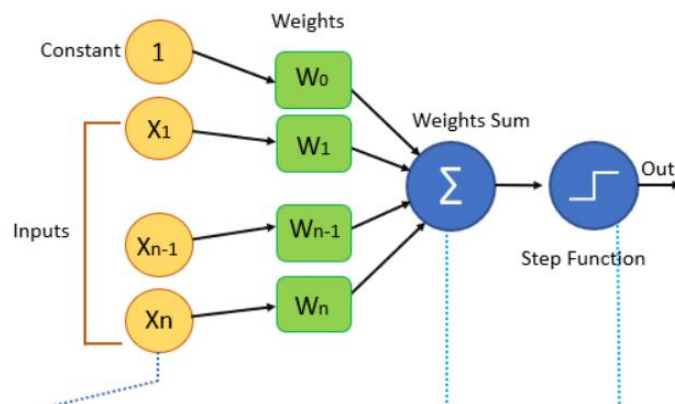
El funcionament simplificat d'una neurona és el següent, rep la informació a través de les dendrites i el cos neuronal (soma) processa la informació, integra els senyals rebuts i decideix si el senyal és transmès per continuar mitjançant el seu axó arribant fins a les ramificacions nervioses que passen aquesta informació a les dendrites d'una altra neurona mitjançant la **sinapsi**. En termes biològics no existeix un contacte directe entre neurones ja que la senyal es transmet mitjançant neurotransmissors químics.

<https://www.youtube.com/watch?v=SBia0Zv82VQ> (vídeo explicatiu de la Sinapsi neuronal).

Depenent de la naturalesa i la intensitat d'aquests senyals d'entrada, el cervell els processa i decideix si la neurona s'ha d'activar ("disparar") o no.



Perceptró



El diagrama següent mostra l'esquema d'un perceptró, és l'arquitectura més simple de xarxa neuronal creada per Frank Rosenblatt (psicòleg americà) el 1957.

Aquesta neurona artificial és lleugerament diferent per la proposada per McCulloch i Pitts. En aquesta nova neurona les **entrades** ($X_1, X_2, \dots, X_n$) i la **sortida** són números i no valors binaris (activat/esactivat) com proposaven McCulloch i Pitts.

Cada **entrada està associada a un pes** ($W_1, W_2, \dots, W_n$) i es calcula la funció lineal sumant la ponderació de les entrades amb els seus pesos ($W_1 \cdot X_1, W_2 \cdot X_2, \dots, W_n \cdot X_n$).

Finalment aquesta suma es passa a una **funció d'activació/escalonada**, que és la que produeix la sortida.

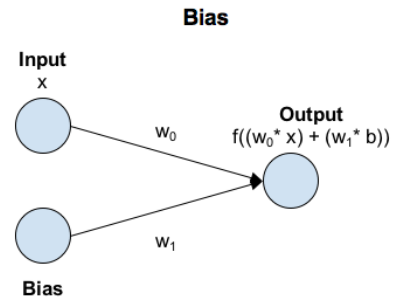
Podríem dir que estem davant d'una funció logística vista en un model de regressió logística.

La **funció d'activació** és el **mecanisme de presa de decisions** de les xarxes neuronals. Aquesta funció produeix una sortida binària, per això a vegades s'anomena funció d'escaló binària (step function). Si el valor és >0 , la classificació assignada és 1 o True. Si el valor és <0 , la classificació assignada és 0 o False.

Bias (biaix)

El biaix permet desplaçar la funció d'activació afegint una constant.

En les xarxes neuronals, el biaix es pot entendre com l'equivalent al terme constant d'una funció lineal, mitjançant el qual la recta es desplaça segons el valor de la constant.

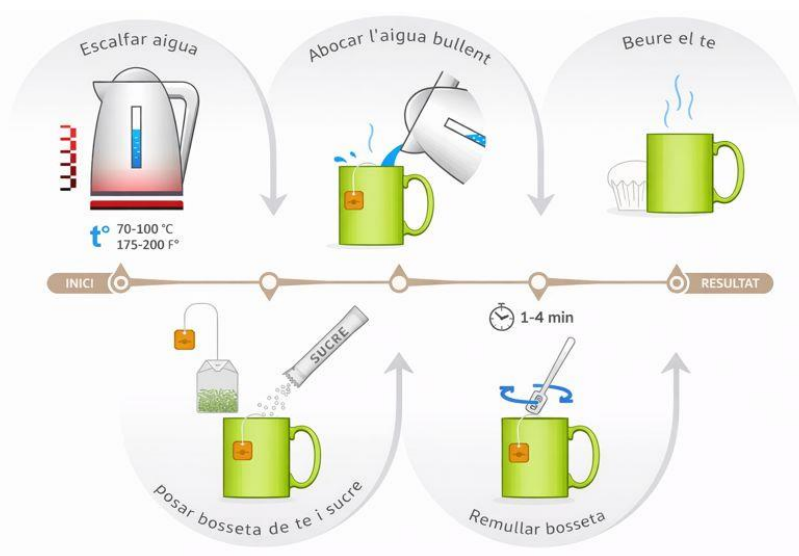


En un escenari amb biaix, l'entrada a la funció d'activació és 'X'

multiplicat pel pes de connexió ' w_0 ' més el biaix multiplicat pel seu pes de connexió ' w_1 '. Això té l'efecte de desplaçar la funció d'activació una quantitat constant : $b \cdot w_1$

Procés d'entrenament d'un perceptró

Considerem l'exemple de "Preparar un te" com a objectiu.



Aquest exemple es pot veure com un perceptró que prepara un bon te.

1. El primer pas és escalfar l'aigua fins arrencar el bull (100 °C) i abocar l'aigua a la tassa.
2. Afegir la bosseta de te i el sucre
3. Esperar 1-4 min. per obtenir el gust i l'aroma perfectes.
4. Finalment, remenar el te i retirar la bosseta.
5. Comprovació de la sortida

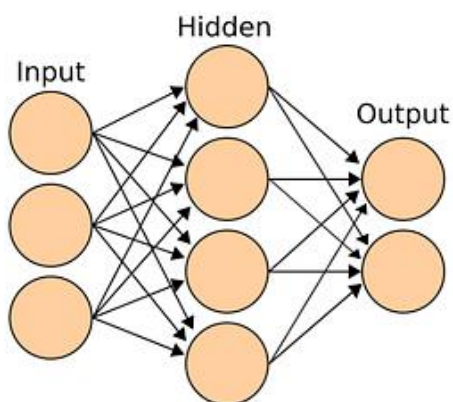
- Si la **sortida** és un **bon te**, no cal fer cap canvi. **FINAL**
- Si la **sortida** és dolenta, cal fer **retropropagació** (*backpropagation*) per canviar la temperatura inicial de l'aigua, la quantitat de sucre, la quantitat d'aigua o el temps d'espera remullant la bossa amb l'aigua. Després es torna a comprovar la sortida (pas 5)

Aquí, l'**entrada** es tracta com l'**aigua** i els **pesos** es consideren com la **quantitat de sucre** i el **temps de la bosseta amb l'aigua**. Cal ajustar els pesos adequadament fins obtenir un **bon te**.

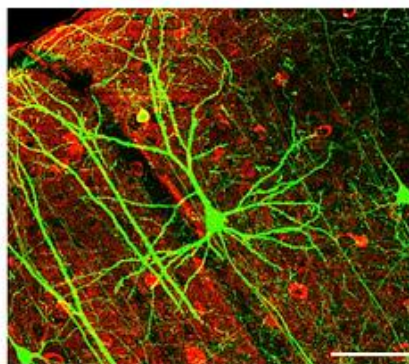
Xarxa neuronal

De la mateixa manera que una sola neurona poca feina pot resoldre. En el cas d'un perceptró passa el mateix. El que es sol fer és unir/connectar diferents perceptrons per resoldre diferents operacions més complexes.

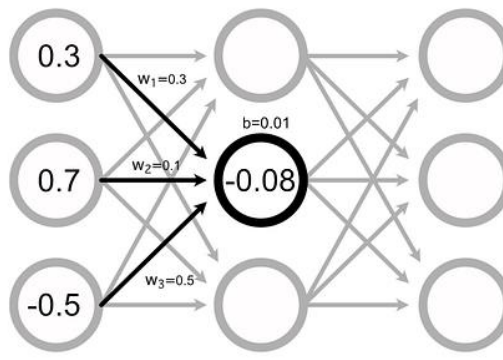
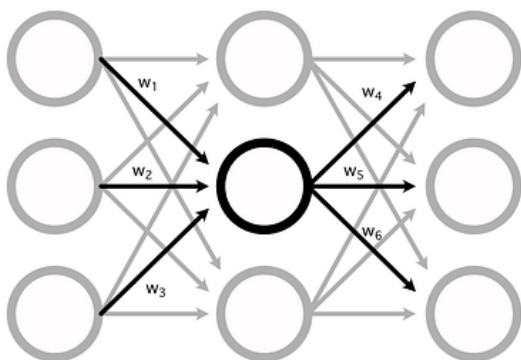
A Neural Network



A Brain



[Xarxa neuronal](#) i [cervell humà](#)



Donades les següents entrades el càlcul de la segona neurona de la segona capa és:

$$resultat = (0.3 \cdot 0.3) + (0.7 \cdot 0.1) + (-0.5 \cdot 0.5) + 0.01 = -0.08$$

Arquitectura bàsica

Com es veu en la representació de les figures anteriors una xarxa neuronal bàsica és la connexió de diferents neurones connectades entre si i organitzades per capes seqüencials.

Capa d'entrada (Input layer)

La capa d'entrada rep les dades brutes del domini. No es realitza cap càlcul en aquesta capa. Els nodes aquí simplement transmeten la informació (característiques) a la capa oculta.

Capa oculta (Hidden layer)

Com el seu nom indica, els nodes d'aquesta capa no estan exposats. Proporcionen una abstracció a la xarxa neuronal. En una xarxa hi poden haver diferents capes ocultes i quan més capes/neurones tingui la xarxa, major serà la seva capacitat per trobar relacions complexes.

La capa oculta **realitza tot tipus de càlculs sobre les característiques introduïdes** a través de la capa d'entrada i transfereix el resultat a la capa de sortida.

Capa sortida (Output layer)

És la capa final de la xarxa és la que porta la informació apresada a través de la capa oculta i, com a resultat, ofereix el valor final.

La capa de sortida normalment utilitzarà una **funció d'activació diferent de la de les capes ocultes**. L'elecció d'aquesta funció depèn de l'objectiu o del tipus de predicció feta pel model.

El flux de la informació segueix una sola direcció, des de la capa d'entrada fins a la capa de sortida passant per les capes ocultes.

Cada neurona està connectada a totes les neurones de la capa següent, això és el que s'anomena xarxa densament connectada o ***full connected***.

Exemple Graus (Cº) a Fahrenheit (Fº):

https://www.youtube.com/watch?v=iX_on3VxZzk&list=PLZ8REt5zt2Pn0vfJjTAPaDVSACDvnuGiG

Funció d'activació

Si ens mirem com es realitza el càlcul d'un perceptró veiem que matemàticament és idèntic a una **regressió lineal**: variables d'entrada multiplicades per uns pesos més un biaix.

$$Y = \beta_1 \cdot X_1 + \beta_0$$

Si la nostra xarxa neuronal fos una combinació de milers de milions d'aquests perceptrons el model resultant continuaria essent només una combinació lineal.

Per solucionar-ho el que fem és utilitzar el que s'anomena una funció d'activació.

La **funció d'activació** és el que **trenca la linealitat** del model canviant així la sortida i aquí rau l'èxit de les xarxes neuronals. D'aquesta manera podem partir l'espai de les dades de forma no lineal

La funció d'activació decideix si una neurona s'ha d'activar o no. Això vol dir que decidirà si l'entrada de la neurona a la xarxa és important o no en el procés de predicció mitjançant operacions matemàtiques simples.

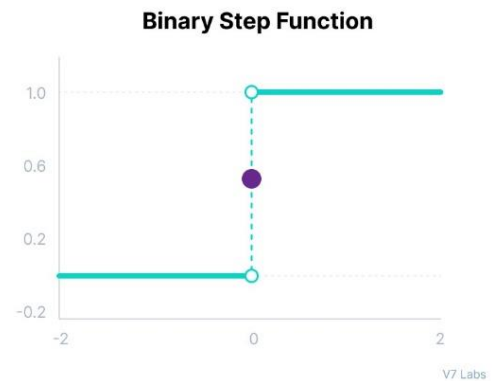
La funció és una mena de filtre que si la sortida no arriba a un cert llindar, no deixarà que surti

Totes les capes ocultes solen utilitzar la mateixa funció d'activació. Tanmateix, la capa de sortida normalment utilitzarà una funció d'activació diferent de la de les capes ocultes. L'elecció depèn de l'objectiu o del tipus de predicció feta pel model.

Podríem pensar que si utilitzem aquestes funcions d'activació estem perdent informació. Això seria així si utilitzéssim una sola neurona, però aquestes funcions d'activació tenen sentit quan combinem diverses neurones amb diverses capes

Funció pas (step)

Aquest tipus de funció és la més simple i actua com un interruptor depenent d'un cert llindar. L'entrada que s'introdueix a la funció es compara amb el llindar i si aquest és superior, la neurona s'activa, en cas contrari es desactiva. Això provoca que la sortida no passa a la següent capa.



Funció sigmoide / logística

Aquesta funció pren qualsevol valor real com a entrada i retorna valors en el rang de 0 a 1.

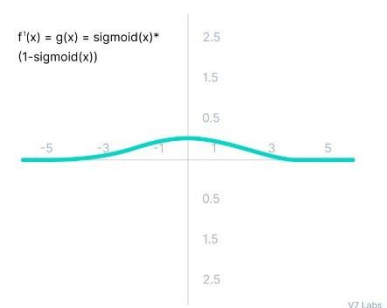
Com més gran sigui l'entrada (més positiva), més a prop estarà el valor de sortida d'1, mentre que com més petita sigui l'entrada (més negativa), més a prop estarà la sortida de 0.

$$f(x) = \frac{1}{1 + e^{-x}}$$

La funció sigmoide és una de les funcions d'activació més utilitzades:

- S'utilitza habitualment per a models on hem de predir la probabilitat com a sortida. Com que la probabilitat de qualsevol cosa només existeix entre el rang de 0 i 1, sigmoide és l'elecció correcta a causa del seu rang.
- La funció és diferenciable i proporciona un gradient suau, és a dir, evita salts en els valors de sortida. Això es representa amb una forma de S de la funció d'activació sigmoide.

La derivada de la funció sigmoide és una funció a on veiem que el rang significatiu va de -3 a 3. Això significa per valors superiors a 3 o inferiors a -3, la funció tindrà gradients molt peits. Això implica que a mesura que el valor del gradient s'acosta a zero, la xarxa deixa d'aprendre.

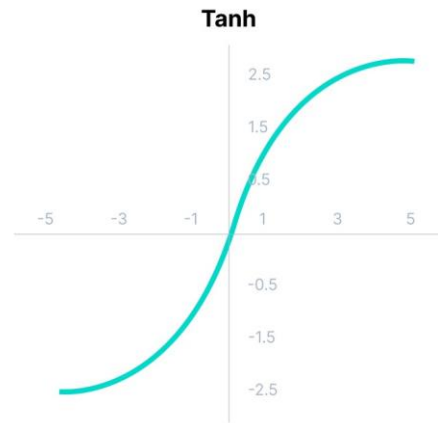


Funció Tanh (tangent hiperbòlica)

La funció Tanh és molt similar a la funció d'activació sigmoide/logística, i fins i tot té la mateixa forma de S, però el seu rang de sortida va de -1 a 1.

En aquesta funció quan més gran sigui l'entrada (més positiva), més a prop estarà el valor de sortida d'1, mentre que com més petita sigui l'entrada (més negativa), més a prop estarà la sortida de -1.

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

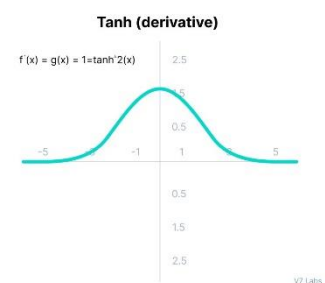


Els avantatges d'utilitzar aquesta funció d'activació són:

- La sortida de la funció d'activació de tanh està centrada en zero; per tant, podem fàcilment mapejar els valors de sortida com a fortament negatius, neutres o fortament positius.
- Normalment s'utilitza en capes ocultes d'una xarxa neuronal, ja que els seus valors es troben entre -1 i 1; per tant, la mitjana de la capa oculta resulta ser 0 o molt propera. **Ajuda a centrar les dades i facilita molt l'aprenentatge per a la següent capa.**

La derivada de la funció tangh pateix el mateix problema que en la sigmoide dels gradients nuls. A més, el gradient de la funció tanh és molt més pronunciat en comparació a la sigmoide

Tot i que tant el sigmoide com el tanh s'enfronten a un problema de gradient nul, **el tanh està centrat en zero** i els gradients no estan restringits a moure's en una determinada direcció. Per tant, a la **pràctica, la no linealitat del tanh sempre es prefereix a la no linealitat del sigmoide**.



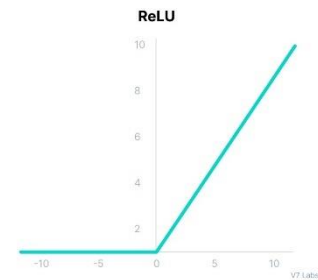
Funció ReLU (Rectified Linear Unit)

Tot i que dona la impressió d'una funció lineal, ReLU té una funció derivada i permet la retropropagació (*backpropagation*) alhora que la fa computacionalment eficient.

El principal inconvenient aquí és que la funció ReLU no activa totes les neurones alhora.

Les neurones només es desactiven si la sortida de la transformació lineal és inferior a 0.

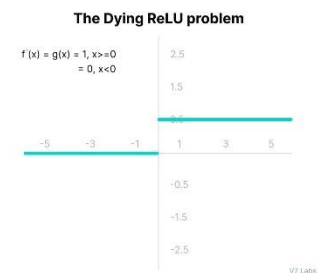
$$f(x) = \max(0, x)$$



Els avantatges d'utilitzar ReLU com a funció d'activació són els següents:

- Com que només s'activen un cert nombre de neurones, la funció ReLU és molt més eficient computacionalment en comparació amb les funcions sigmoide i tanh.
- ReLU accelera la convergència del descens de gradient cap al mínim global de la funció de pèrdua.

Les limitacions a què s'enfronta aquesta funció és principalment el problema Dying ReLU. El costat negatiu del gràfic fa que el valor del gradient sigui zero. Per aquest motiu, durant el procés de retropropagació, els pesos i els biaixos d'algunes neurones no s'actualitzen. Això pot crear neurones mortes que mai s'activen.

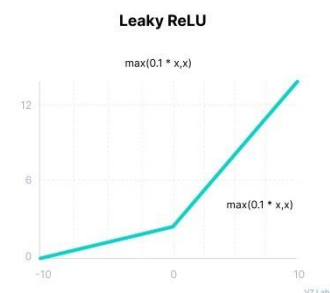


Tots els valors d'entrada negatius esdevenen zero immediatament, cosa que disminueix la capacitat del model per ajustar o entrenar correctament a partir de les dades.

Funció Leaky ReLU

Leaky ReLU és una versió millorada de la funció ReLU per resoldre el problema Dying ReLU, ja que té un petit pendent positiu a la zona negativa.

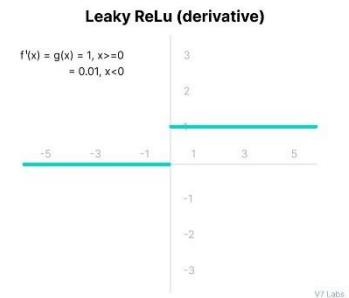
$$f(x) = \max(0.1x, x)$$



Els avantatges de Leaky ReLU són els mateixos que els de ReLU, a més del fet que permet la retropropagació (*backpropagation*), fins i tot per a valors d'entrada negatius.

Fent aquesta petita modificació per a valors d'entrada negatius, el gradient del costat esquerre del gràfic resulta ser un valor diferent de zero. Per tant, ja no trobaríem neurones mortes en aquesta regió.

La derivada de la funció Leaky ReLU.



Les limitacions a què s'enfronta aquesta funció inclouen:

- És possible que les prediccions no siguin consistents per a valors d'entrada negatius.
- El gradient per a valors negatius és un valor petit que fa que l'aprenentatge dels paràmetres del model requereixi molt de temps.

Funció Softmax

En la teoria de la probabilitat, la sortida de la funció softmax es pot utilitzar per representar una distribució categòrica, és a dir, una distribució de probabilitat sobre K resultats possibles diferents. Per tant, la funció softmax s'utilitza per a la classificació multiclasse i s'utilitza a la última capa.

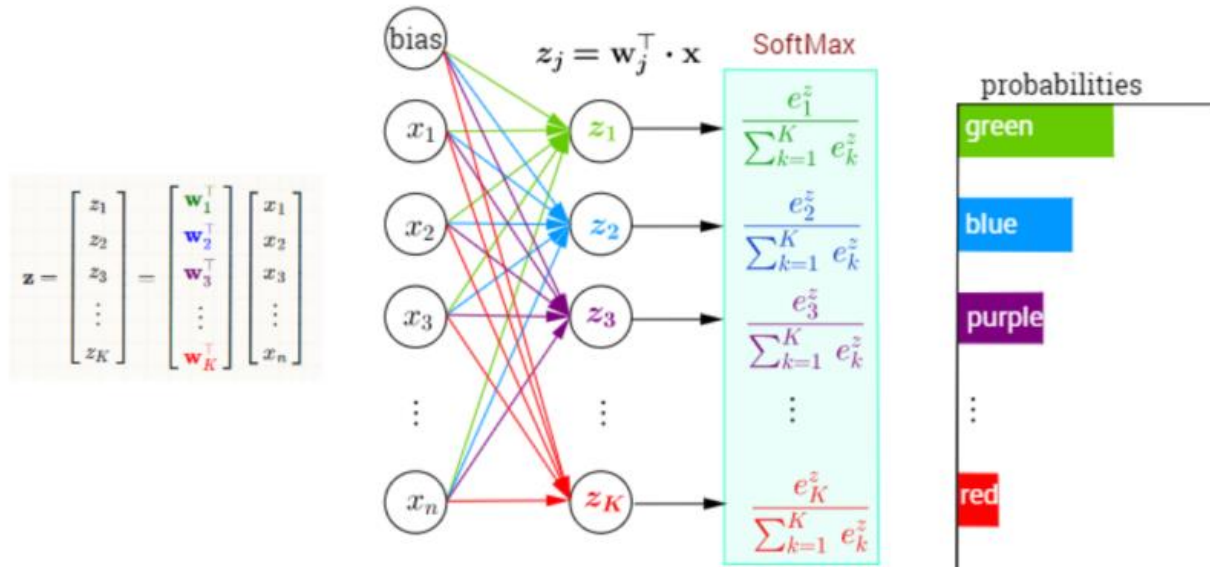
Podem considerar la funció softmax com una generalització de la funció sigmoide que permet classificar més de dues classes. La funció d'activació softmax ens garanteix que totes les probabilitats estimades es troben entre 0 i 1 i que la seva suma és igual a 1.

A nivell matemàtic, softmax utilitza el valor exponencial de les evidències calculades i, posteriorment, les normalitza de manera que sumin 1, formant així una distribució de probabilitat.

$$\sigma : \mathbb{R}^K \rightarrow [0, 1]^K$$
$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

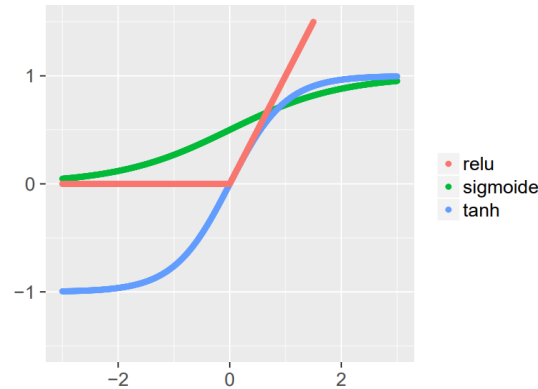
Exemple de la funció d'activació softmax:

Multi-Class Classification with NN and SoftMax Function



Quadre resum de les funcions d'activació





Name	Visualization	$f(x) =$	Notes
Linear (= Identity)		x	Not useful for hidden layers
Heaviside Step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	Not differentiable
Rectified Linear (ReLU)		$\begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	Surprisingly useful in practice
Tanh		$\frac{2}{1+e^{-2x}} - 1$	A soft step function; ranges from -1 to 1
Logistic ('sigmoid')		$\frac{1}{1+e^{-x}}$	Another soft step function; ranges from 0 to 1

Resum

- Les funcions d'activació s'utilitzen per introduir no linealitat a la xarxa.
- Una xarxa neuronal gairebé sempre tindrà la mateixa funció d'activació en totes les capes ocultes. Aquesta funció d'activació hauria de ser diferenciable per tal que els paràmetres de la xarxa s'aprenuin en retropropagació.
- **ReLU és la funció d'activació més utilitzada per a capes ocultes.**
- Es busquen funcions que les seves derivades siguin simples per minimitzar el cost computacional.
- A l'hora de seleccionar una funció d'activació, cal tenir en compte els problemes que podria afrontar: gradients que desapareixen i exploten.
- Pel que fa a la capa de sortida, sempre hem de considerar el rang de valors esperats de les prediccions. Si pot ser qualsevol valor numèric (com en el cas del problema de regressió) podeu utilitzar la funció d'activació lineal o ReLU.
- Utilitzeu la funció **Softmax** o Sigmoid per als problemes de **classificació**.

- El fet de posar funcions no lineals dona expressivitat de la xarxa. Per aquest motiu una xarxa neuronal pot fer overfitting molt fàcilment si afegim moltes capes i neurones ja que tindrà tants paràmetres que trobarà una combinació de paràmetres assignarà 1 a 1 l'entrada i la sortida.

Capa	Funció activació
Cap d'entrada	Cap
Capa oculta	ReLU / Tanh/etc..
Capa sortida (binària)	Sgimoide
Capa sortida (multiclasse)	Softmax
Capa sortida (regressió)	Identitat

Entrenament

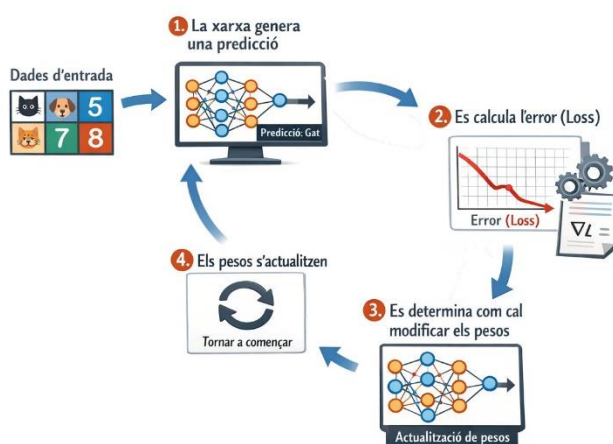
El procés d'entrenament el que farà serà anar **modificant els paràmetres interns** (pesos i bias) de **cadascuna de les neurones** perquè el model produeixi prediccions cada vegada més bones.

Una xarxa neuronal **no sap res a priori abans de començar** l'entrenament. Els **pesos** de les neurones **s'inicialitzen amb valors aleatoris** i, a partir d'aquí, el model millora progressivament observant els resultats i anar **minimitzant la funció de Loss**.

Aquest procés d'aprenentatge es basa en un

bucle que es repeteix moltes vegades:

1. Les dades entren a la xarxa
2. La xarxa genera una predicció
3. Es calcula l'error (loss)
4. Es determina com cal modificar els pesos
5. Els pesos s'actualitzen
6. El procés es repeteix



Aquest cicle és el cor de l'aprenentatge de les xarxes neuronals on hi trobem 4 peces fonamentals:

- Forward propagation
- Funció de pèrdua/cost (Loss)
- Backpropagation
- Gradient descent (Descens de gradient)

Forward propagation

El **forward propagation** és el pas "mecànic": **agafes una entrada i la fas passar per tota la xarxa capa a capa fins obtenir una predicció**. Aquí no hi ha aprenentatge: només càlcul. **En cada capa**, el model **aplica la transformació lineal + activació** i produeix la següent representació.

Podríem veure'l com una cadena de muntatge a on cada capa rep un producte semielaborat i el transforma per passar-lo a la següent capa. Al final, l'última capa produeix la predicció (una classe, una probabilitat o un número).

Després del forward, calculem la pèrdua/loss, que és el "veredict" del model per aquella predicció.

Funció de cost (loss function)

La **funció de cost** és la mètrica que tenim per dir-li al model “com de malament/bé ho està fent”. Sense una loss, el model podria produir números, però no tindria manera de saber si millora o empitjora. En l’entrenament, l’objectiu és trobar pesos i bias que **minimitzin** aquesta funció.

Regressió

En una regressió, la Loss típica és el **Mean Squared Error (MSE)**, que penalitza més fort els errors grans perquè eleva al quadrat:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Si recordes, en les mètriques de regressió, l’MSE “castiga molt” els *outliers*. Si tenim un dataset de preus i hi ha un pis de luxe que el model prediu molt malament, el quadrat farà que aquest error tingui un impacte desproporcionat.

Una alternativa és el **Mean Absolute Error (MAE)**, que és més robust:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Classificació

En classificació, la loss més habitual és la **cross-entropy** (log loss). El model ha de donar probabilitat alta a la classe correcta. Si el model dona 0.99 a la classe correcta, “bé”; si dona 0.01, “fatal”. La cross-entropy penalitza molt els casos en què el model està segur però equivocat. Això és ideal perquè “educa” el model a calibrar probabilitats.

En binària, es formula com:

$$L = -(y \log(p) + (1 - y) \log(1 - p))$$

On p és la probabilitat predita de classe 1. En multiclasse, s’estén sumant sobre classes.

La regla professional és: tria la loss que encaixa amb la interpretació de sortida.

Sortida probabilística → CROSS-ENTROPY

Sortida numèrica → MSE/MAE.

Backpropagation (retropropagació de l'error)

El backpropagation és la clau de l'aprenentatge. La intuïció que funciona millor és: “Un cop tens l'examen corregit (la loss), has de repartir la responsabilitat de l'error entre totes les decisions que has pres”. En una xarxa, aquestes “decisions” són els pesos i els biaixos. La pregunta és: **quins pesos han contribuït més a l'error?**

El procés comença a la capa de sortida a on coneixem l'error/loss, i propaguem cap enrere fins arribar a la capa d'entrada i anar ajustant els pesos de cada neurona per la qual anem passant.

Matemàticament, això es fa amb derivades i mitjançant el **descens de gradient**.

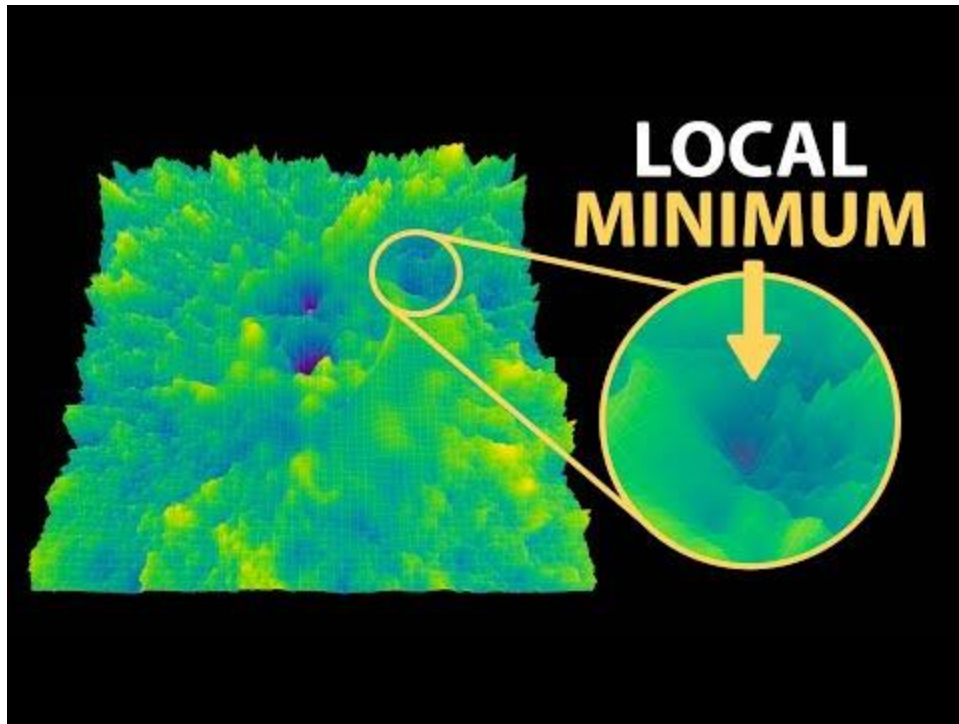
Gradient descent (Descens de gradient)

Un cop tenim definida la funció de cost el nostre objectiu és trobar els paràmetres del nostre model que minimitzin aquesta funció.

El **gradient** indica cap a la **direcció** on la **loss creix més ràpid**. Per tant, si volem reduir la loss, hem d'anar en direcció contrària al gradient.



Amb això té sentit parlar de la **derivada**, ja que la derivada d'una funció en permet saber el comportament d'una funció. Per exemple ens permet saber en quins intervals una funció creix o decreix i amb quin ritme/rapidesa ho fan.



<https://www.youtube.com/watch?v=NrO20Jb-hy0>

Optimització i actualització de pesos

L'**optimitzador** és un component clau d'una xarxa neuronal responsable d'actualitzar els paràmetres del model durant el procés d'entrenament. Concretament, la seva funció és **minimitzar la funció de pèrdua** de la xarxa neuronal ajustant els **pesos** i els **biaixos** del model.

Durant l'entrenament, la xarxa neuronal rep dades d'entrada i produeix una sortida basada en els seus paràmetres actuals. Aquesta sortida es compara amb la sortida desitjada (és a dir, la veritat de referència o ground truth) mitjançant una funció de pèrdua, que mesura com de bé està funcionant el model. L'**optimitzador** calcula aleshores els gradients de la pèrdua respecte als paràmetres del model i utilitza aquests gradients per actualitzar-los en la direcció que redueix la pèrdua.

L'optimitzador és qui decideix com han de canviar els pesos en cada pas de l'entrenament

L'elecció de l'optimitzador pot tenir un impacte significatiu en el rendiment de la xarxa neuronal, ja que cada optimitzador té avantatges i inconvenients diferents. Alguns optimitzadors populars són:

- **Descens del gradient estocàstic (SGD):** és un optimitzador senzill que actualitza els paràmetres en la direcció del gradient negatiu de la funció de pèrdua.
- **Adam (Adaptive Moment Estimation):** és un optimitzador més sofisticat que combina taxes d'aprenentatge adaptatives i *momentum* (*vola baixant per una vall, si sempre baixa en la mateixa direcció accelera*) per convergir de manera més ràpida i robusta. Ajusta automàticament la velocitat d'aprenentatge per cada pes.
- **Adagrad:** aquest optimitzador adapta la taxa d'aprenentatge de cada paràmetre basant-se en la informació històrica dels gradients.
- **RMSprop:** també adapta la taxa d'aprenentatge de cada paràmetre, però utilitza una mitjana mòbil del gradient al quadrat per fer-ho.
- **Adadelta:** és similar a RMSprop, però utilitza un mètode més sofisticat per adaptar la taxa d'aprenentatge que no requereix definir explícitament aquest paràmetre.

En conjunt, l'optimitzador és un element fonamental en el procés d'entrenament d'una xarxa neuronal, ja que ajuda el model a aprendre el conjunt òptim de paràmetres per a la tasca que s'ha de resoldre.

Regularització

En l'**aprenentatge automàtic**, la **regularització** és un conjunt de tècniques utilitzades per prevenir el **sobreajustament** (*overfitting*) en els models, incloses les xarxes neuronals. L'*overfitting* es produeix quan un model esdevé massa complex i comença a ajustar-se al soroll de les dades d'entrenament en lloc de capturar els patrons. Això pot provocar un rendiment deficient sobre dades noves que el model no ha vist abans. Les tècniques de regularització ajuden a evitar el sobreajustament limitant la capacitat del model o afegint restriccions addicionals al procés d'optimització.

En les xarxes neuronals, la regularització es pot aplicar de diverses maneres. Algunes de les tècniques més habituals són:

- **Regularització L1 (Lasso), L2 (Ridge) i Elastic Net:** Aquestes tècniques afegeixen un terme de penalització a la funció de pèrdua que incentiva que els pesos del model siguin petits.
- **Dropout:** Aquesta tècnica elimina aleatòriament algunes neurones de la xarxa durant l'entrenament, obligant les neurones restants a aprendre representacions més robustes i diverses de les dades.

- **Aturada anticipada (*Early stopping*):** Aquesta tècnica atura el procés d'entrenament abans que el model s'ajusti completament a les dades d'entrenament, evitant el sobreajustament mitjançant la recerca d'un equilibri òptim entre infrajustament (*underfitting*) i sobreajustament (*overfitting*). Aturem quan la validació empitjora.
- **Augmentació de dades (*Data augmentation*):** Aquesta tècnica incrementa la mida del conjunt d'entrenament aplicant transformacions aleatòries a les dades d'entrada, com ara rotacions, retalls o inversions d'imatges. Això ajuda el model a ser més robust davant variacions en les dades d'entrada.

La **regularització** és una **part essencial del procés d'entrenament** de les xarxes neuronals, ja que **ajuda a prevenir el sobreajustament** i a **millorar la capacitat de generalització** del model. L'elecció de la tècnica de regularització depèn del problema específic que es vol resoldre i de les característiques del conjunt de dades.

Preprocessat

En dades tabulars, és habitual:

- **Escalat/estandardització** de variables numèriques (StandardScaler o MinMaxScaler)
- **One-hot encoding** de categòriques (si és tabular tradicional)

L'escalat és clau perquè si una feature va de 0 a 1 i una altra va de 0 a 100000, la segona dominarà els gradients i l'entrenament serà inestable. És com fer una votació on una persona té 100.000 vots i l'altra en té 1: no és "just" per al model.

Disseny de l'arquitectura

Hiperparàmetres

Les xarxes tenen molts hiperparàmetres: nombre de capes, neurones, activació, learning rate, batch size, regularització, optimitzador... i tots interactuen. Això és important per fer entendre que "provar un hiperparàmetre aïllat" sovint no és suficient; cal fer recerca conjunta (GridSearch/RandomSearch o mètodes més moderns). Per FP, la idea essencial és: **més capacitat (més neurones i capes) no sempre és millor.**

Frameworks de Xarxes Neuronals

De la mateix manera que scikit-learn és una de les llibreries a on trobem infinitat de models de Machine Learning. Inclús un perceptró, existeixen altres llibreries i frameworks més especialitzats. Els més populars actualment són TensorFlow, PyTorch i Keras.

TensorFlow

TensorFlow és la llibreria de *Deep Learning* més popular i amb més contribucions. Va ser **desenvolupada** originalment per **Google Brain** per a l'ús intern en productes d'IA de Google i es va fer **open source** el 2015 i des de llavors s'ha convertit en l'estàndard de facto.

TensorFlow és la base de productes de Google com Google Translate i les API de machine learning de Google Cloud Platform.



TensorFlow es va dissenyar perquè es pugues paral·lelitzar, i per això té un rendiment excel·lent en entorns distribuïts.

TensorFlow proporciona **API** per a **Python, C++, Java i altres llenguatges**

En Python es pot instal·lar fàcilment mitjançant pip

```
pip install tensorflow
```

PyTorch

PyTorch és una llibreria *Deep Learning* creada pel laboratori de recerca en intel·ligència artificial de Facebook (META). A diferència de TensorFlow, PyTorch està dissenyat per imitar Python natiu, fet que el fa molt intuïtiu per a programadors de Python.



Es pot instal·lar amb:

```
conda install pytorch torchvision -c pytorch
```

Keras

Keras és la llibreria de Deep Learning de més alt nivell. Actualment està integrada dins TensorFlow i es considera el wrapper per excel·lència de TensorFlow a partir de la versió 2.4, però en principi pot funcionar sobre altres llibreries com MXNet o Theano. Es va publicar com un projecte de codi obert el 2015, però va ser desenvolupat per François Chollet com a part d'un procés d'investigació. Quan instal·lem TensorFlow també s'inst·la Keras de forma automàtiques.



Es pot instal·lar amb:

```
pip install keras
```

Exeprimentació amb TensorFlow

[TensorFlow Playground](#) és una aplicació web de visualització interactiva, escrita en JavaScript, que ens permet simular xarxes neuronals densament connectades que s'executen al nostre navegador i veure els resultats en temps real.

Permet afegir fins a 6 capes internes amb fins a 8 neurones per capa. En entrenar la xarxa neuronal, podem veure si ho estem aconseguint o no mitjançant la mètrica "Training loss", és a dir, la funció de pèrdua per a les dades d'entrenament. Posteriorment, per comprovar que el model generalitza correctament, també s'ha d'aconseguir minimitzar la "Test loss", és a dir, l'error calculat per la funció de pèrdua per a les dades de test.

Es pot jugar amb aquest simulador realitzant els següents canvis:

- Augmenta la proporció de dades d'entrenament respecte a les de test fins al 70%
- Deixa una única capa interna amb una sola neurona
- Afegeix 2 neurones més a la capa interna (de manera que tingui 3 neurones)

- Canviar el conjunt de dades pel que té 4 zones quadrades diferents
- Canviar el conjunt de dades pel que té forma de remolí (necessitarà més capes amb més neurones)
- Canviar els valors dels hiperparàmetre (batch size, learning rate, funcions d'activació regularització,...)
- Afegir soroll a les dades d'entrada

Conclusions:

- Poques neurones a les capes ocultes provocaran infrajustament (*underfitting*).
- Massa neurones a les capes ocultes provocaran sobreajustament (*overfitting*) —la xarxa neuronal té més capacitat de processament d'informació que la quantitat d'informació continguda en el conjunt d'entrenament, que no és suficient per entrenar totes les neurones de les capes ocultes— i també un temps de processament molt més elevat.

Implementació d'una xarxa neuronal amb Keras

Keras és l'API de Deep Learning a al nivell de TensorFlow. Permet crear, entrenar, avaluar i executar tot tipus de xarxes neuronals.

Exemple de Xarxa Neuronal - Regressió

Tipus de Xarxes Neuronals

FNN /MLP (Feedforward Neural Network /Multilayer Perceptron)

Una xarxa neuronal directa (FNN) és el tipus més bàsic de xarxa neuronal artificial en l'aprenentatge automàtic.

La informació flueix en una direcció, des de l'entrada fins a la sortida, sense cap bucle/cicle o retroalimentació.

Per què podem utilitzar les xarxes FNN?

- **Aproximador universal de funcions:** Les xarxes neuronals *feedforward* (FNN) poden aproximar qualsevol funció contínua, cosa que les fa molt potents tant per a tasques de **regressió** com de **classificació**.
- **Reconeixement de patrons:** Són molt eficients aprenent patrons dins les dades, com ara en imatges, text o dades estructurades.
- **Generalització:** Un cop entrenades, generalitzen bé a dades no vistes, sobretot si aquestes són similars a les dades amb què s'han entrenat.

Quan utilitzar-les?

- **Classificació:** Quan necessitem classificar dades en diferents categories (p. ex., categories de productes en una botiga minorista).
- **Regressió:** Quan necessitem predir valors continus (p. ex., predir vendes a partir del rendiment passat).
- **Extracció de característiques:** Quan necessitem aprendre relacions entre característiques d'entrada (p. ex., relacions entre la demografia dels clients i el seu comportament de compra).

Les FNN són adequades en els següents escenaris:

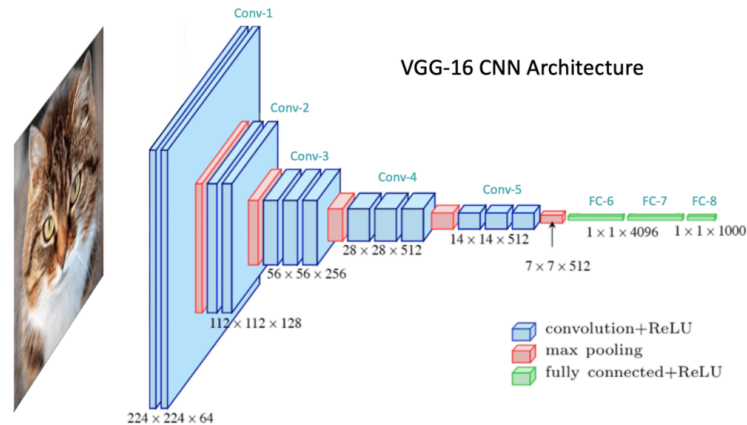
- Problemes d'aprenentatge supervisat: tasques com classificació (p. ex., identificar segments de clients) i regressió (p. ex., predir vendes futures).
- Dades estructurades: quan tenim dades tabulars, com registres de transaccions de clients, dades de vendes, etc.

- Reconeixement de patrons bàsic: quan les relacions entre entrada i sortida són relativament directes (p. ex., predir els ingressos d'una botiga a partir del trànsit de persones).

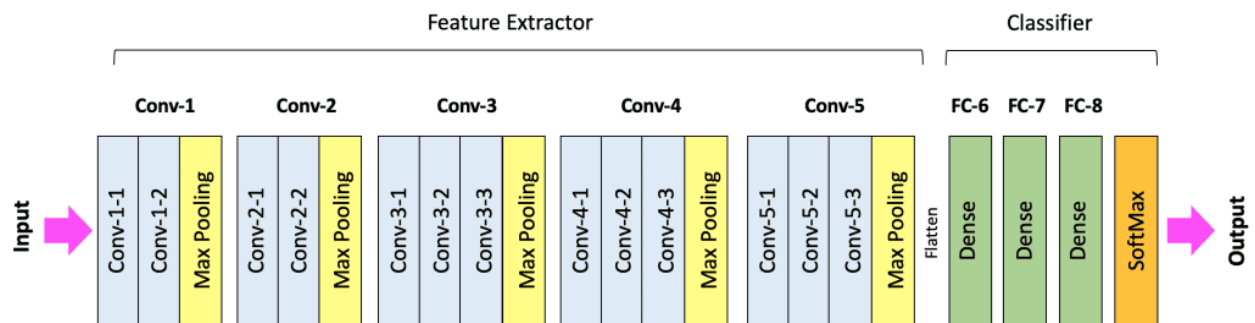
CNN (Convolutional Neural Network)

Capes de convolució

Popularment s'utilitzen per tractament d'imatges.



<https://learnopencv.com/>



Vídeo explicatiu de com funciona una xarxa convolucional 2D: <https://youtu.be/yb2tPt0QVPY>

RRNN (

Xarxa neuronal Afegir algunes connexions amb la sortida per tenir una mica de feedback. Això es fa per dotar a la xarxa de certa memòria. Per processar seqüències.

Processar un text, ...

LSTM

És un tipus especial de RRN. Tenen la característica per solucionar un problema que tenen les xarxes neuronals que és la disminució del gradient. Aquesta senyal es va perdent la senyal de gradient.

Vanish gradients...

GANs

És una xarxa dissenyada per generar no per classificar. Tenen 3 blocs una és la xarxa que volem que digui les coses una altra que processa les nostres dades reals i una tercera que combina les dues.

Volem generar cares. Generam una sortida (CNN) aleatori, però tenim una altra xarxa FCN que intenta quan jo li passo una imatge real la xarxa detecta si és real o generada per mi. Aquest error anem actualitzant la que genera la imatge perquè generi menys error i sigui més fidedigne. A final arriba un punt que el discriminant ja no distingeix entre la imatge real o de la generada. I al final tenim un model capaç de generar imatges.

Feaix això és molt maco, però fer una xarxa d'aquest tipus i que funcioni no és fàcil. Ja que tenim un generador i un discriminador.

Xarxes Autoencoder

Tota xarxa és un encoder perquè un encoder si tenim una xarxa a l'entrada tenim un vector amb 3 valors i amb aquest vector el transformen en un altre vector de dimensió 2. Això és un embedding. 3.000 entrades i passar a 100. Espai d'alta dimensió i el passem a un espai de més baixa dimensió.

Dissenyat per aprendre una representació comprimida de les dades d'entrada, que després es pot utilitzar per a tasques com ara la reducció de soroll d'imatges, la compressió de dades i la detecció d'anomalies. Els autocodificadors consten d'una xarxa de codificadors que comprimeix les dades d'entrada i una xarxa de decodificadors que intenta reconstruir les dades originals a partir de la representació comprimida.

Referències

- Aurélien Géron (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- Patrick D. Simith(2018). Hands-On Artificial Intelligence for Beginners. Packt
- Continguts de la docent Cristina Gómez de IES de Teis Vigo
- Article [*What is Perceptron: A Beginners Tutorial For Perceptron*](#), Antony Christopher
- Funcions activació:
 - <https://www.v7labs.com/blog/neural-networks-activation-functions>
 - <https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/>
- Backpropagation: [*Código Máquina*](#)
- <https://cienciadedatos.net/documentos/py35-redes-neuronales-python>
- CNN (Convolutional Neural Network) : <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>
- Gradient descent:
 - <https://www.youtube.com/watch?v=za61eVtq2MY>
 - <https://www.youtube.com/watch?v=za61eVtq2MY&t=459s>
-
- Keras Cheat Sheet: <https://www.datacamp.com/cheat-sheet/keras-cheat-sheet-neural-networks-in-python>